INSTITUTO SUPERIOR DE ENGENHARIA DO PORTO

Tópicos de Matemática para Sistemas Autónomos (TOMSA 2026)

Especificação de Requisitos, Diagnóstico dos Dados e Arquitetura de Software Unificada para o Visualizador Multi-Robô

Consolidação dos Subsistemas Plotly 3D, Vista em 1.ª Pessoa (POV), Filtro Kalman/RTS, Alinhamento Kabsch-Umeyama e Análise de Proximidade CPA

Geovani Lopes Debastiani (Nº 1211756)
1211756@isep.ipp.pt

5 de outubro de 2026

Resumo

Este documento estabelece a consolidação formal dos requisitos, o diagnóstico técnico das anomalias encontradas nas telemetrias reais do INESC TEC (Aguçadoura) e a especificação da arquitetura de software unificada para o sistema multi-robô subaquático da unidade curricular de TOMSA (2026). Atualmente, a base de código encontra-se fragmentada em três implementações parciais: o projeto de referência `TOMSA`, o visualizador estático 3D `trajectories` e o módulo de câmara em primeira pessoa `vista_primeira_pessoa`. O objetivo primordial consiste em fundir organicamente estas soluções num único projeto modular, robusto, multithreaded e centrado numa interface web interativa em Plotly/Dash. Abordam-se os requisitos funcionais de suavização cinemática via Filtro de Kalman com suavizador RTS (*Rauch-Tung-Striebel*), alinhamento rígido de referenciais pelo algoritmo de Kabsch-Umeyama, análise de segurança e proximidade (AER, CPA e deteção de colisões), bem como a exportação geoespacial (KML e GeoJSON).

Conteúdo

1 Introdução e Contexto do Projeto **2**
1.1 Estado Atual e Fragmentação de Código 2

2 Diagnóstico Técnico das Discrepâncias e Dados Reais **2**

3 Catálogo Estruturado de Requisitos **3**
3.1 Requisitos Funcionais (RF) 3
3.2 Requisitos Não-Funcionais e Arquiteturais (RNF) 4

4 Arquitetura de Software Proposta **4**
4.1 Estrutura de Ficheiros e Módulos do Novo Projeto 5
4.2 Fluxo de Dados e Concorrência Multithreaded 6

5 Roteiro de Implementação e Validação **6**

1 Introdução e Contexto do Projeto

O projeto tem como finalidade primordial o processamento, sincronização, análise geométrica e visualização tridimensional de veículos marinhos autónomos (AUVs e ASVs) com base em registos reais de odometria (`nav_msgs/Odometry` do ecossistema ROS) recolhidos pelo INESC TEC em campanhas marítimas na Aguçadoura (`robot1_s1.csv`, `robot4_s1.csv`, `robot7_s1.csv`).

1.1 Estado Atual e Fragmentação de Código

Na estrutura existente, coexistem três ramificações isoladas:

1. **Projeto Base (`TOMSA`)**: Contém a formulação matemática rigorosa de coordenadas (Geodésico WGS-84, ECEF, NED, ENU, AER), classes de dados estruturadas, configuração YAML e visualização interativa em Matplotlib (2D/3D com slider temporal).
2. **Visualizador 3D Plotly (`trajectories / uwv`)**: Gera um ficheiro HTML interativo autónomo em Plotly com os rastos dos 3 robôs em coordenadas NED locais e profundidade (D), permitindo rotação e inspeção 3D fluida.
3. **Câmara em 1.ª Pessoa (`vista_primeira_pessoa`)**: Implementa um servidor web local com projeção de câmara *pinhole* a partir do referencial do corpo de cada drone (POV – *Point of View*), permitindo alternar de ponto de vista através das teclas 1, 2, 3, mas operando com discrepâncias de origem e orientação.

O utilizador estipulou a necessidade imperativa de unificar todas estas capacidades numa única arquitetura integrada e de alta performance, utilizando o **Plotly** como visualizador principal e incorporando ferramentas matemáticas avançadas de suavização, alinhamento e análise de proximidade.

2 Diagnóstico Técnico das Discrepâncias e Dados Reais

O utilizador reportou que os ficheiros `robotX_s1.csv` aparentavam estar "corrompidos" ou não coincidir entre os visualizadores. A análise minuciosa aos dados revelou que os dados não estão corrompidos, mas sofrem de quatro causas matemáticas e operacionais objetivas:

Discrepância	Causa Raiz Identificada	Solução na Arquitetura Unificada
1. Desvio de Origem (p_0)	A pasta `trajectories` calcula a origem pela média de todas as amostras ($\bar{\varphi}, \bar{\lambda}, \bar{h}$); a pasta `vista_primeira_pessoa` usa a média dos robôs carregados; o projeto `TOMSA` usa a origem oficial de superfície do INESC TEC ($\varphi_0 = 41.452017°, \lambda_0 = -8.813222°, h_0 = 55.447911\,\text{m}$).	Unificar num único ponto de referência c figurável via `config.yaml`, garantindo eq valência milimétrica em todas as vistas.
2. Assincronismo Temporal	Cada robô foi amostrado a instantes desfasados (período médio $\Delta t \approx 0.534\,\text{s}$), com instantes de início e fim distintos (janela comum entre $t \approx 1.626 \times 10^9$ s).	Criar uma grelha temporal mestre regula zada, aplicando Filtro de Kalman e suavi dor RTS para alinhar perfeitamente os es dos.
3. Inversão do Robot 7	O Robot 7 apresenta quaternions de orientação rodados em $\sim 180°$ em relação ao vetor velocidade de avanço (marcha-atrás ou referencial local ROS com convenção invertida no sensor).	Implementar deteção automática de coer cia cinemática (ângulo entre $\mathbf{v}$ e o eixo lon tudinal x_b) e aplicar correção controlada p parâmetro.
4. Mapeamento de Eixos	O referencial NED usa z para baixo, enquanto o motor 3D do Plotly adota convenção gráfica (x direita, y frente, z cima).	Mapeamento isométrico formal: $X_{\text{Plotly}} = Y_{\text{Plotly}} = N$, $Z_{\text{Plotly}} = -D = h - h_0$.

Tabela 1: Matriz de diagnóstico das discrepâncias entre os módulos existentes.

3 Catálogo Estruturado de Requisitos

3.1 Requisitos Funcionais (RF)

RF1: Visualização Principal em Plotly 3D: A interface central do sistema deve ser construída em Plotly/Dash (ou HTML5 interativo gerado por Plotly), suportando navegação espacial completa (rotação, pan, zoom e vistas axiais).

RF2: Modo Global View: Representação simultânea das trajetórias de todos os robôs no referencial de missão comum (NED/ENU), com diferenciação de cores, marcadores de início/fim e indicação da posição em tempo real.

RF3: Modo POV / 1.ª Pessoa (Local Body View): Capacidade de assumir a perspetiva de qualquer um dos submarinos, posicionando a câmara na pose do veículo e projetando os outros robôs no campo de visão relativo, alternando interativamente entre veículos (teclas 1, 2, 3 ou seletor).

RF4: Replay Temporal Interativo: Controlo da reprodução com barra temporal deslizante (*time slider*), botões *Play/Pause*, ajuste de velocidade ($1\times, 2\times, 5\times, 10\times$) e salto para instantes específicos.

RF5: Filtragem e Suavização Cinemática (Kalman + RTS Smoother): Implementação de um Filtro de Kalman linear/cinemático acoplado a um suavizador de Rauch-Tung-Striebel (RTS), eliminando ruído de GPS/odometria e reconstruindo estados contínuos de posição e velocidade.

RF6: Alinhamento de Referenciais (Kabsch-Umeyama): Módulo capaz de estimar a rotação ótima $\mathbf{R} \in SO(3)$, translação $\mathbf{t} \in \mathbb{R}^3$ e fator de escala $s \in \mathbb{R}^+$ entre trajetórias expressas em referenciais não coincidentes, quantificando o RMSE do alinhamento.

RF7: Análise de Proximidade e Segurança (AER, CPA, Colisão): Cálculo contínuo da distância relativa entre todos os pares de robôs, determinação de Azimute, Elevação e Alcance (AER) na vista local, cálculo do Ponto de Maior Aproximação (*Closest Point of Approach* – CPA) e emissão de alertas gráficos de perigo de colisão.

RF8: Tabela Comparativa de Interpoladores: Módulo para comparação quantitativa de interpolação Linear, Spline Cúbica e Filtro de Kalman/RTS sobre os mesmos dados de telemetria, reportando métricas de erro (RMSE, erro máximo e continuidade).

RF9: Exportação Geoespacial (KML e GeoJSON): Exportação direta das trajetórias processadas e eventos de aproximação para ficheiros `.kml` e `.geojson`, permitindo abertura direta no Google Earth e no QGIS.

RF10: Configuração Unificada (YAML): Todas as propriedades da missão (origem geodésica, ficheiros de dados, limiares de alarme, parâmetros do filtro e opções de visualização) devem ser parametrizáveis via ficheiro YAML sem necessidade de alteração de código.

3.2 Requisitos Não-Funcionais e Arquiteturais (RNF)

RNF1: Separação Estrita de Responsabilidades: Organização em pastas temáticas independentes: matemática (`core/math`), carregamento e filtragem de dados (`core/data`), visualização Plotly e POV (`visualization`), servidor/threads (`server`), e controlo/-CLI principal (`main.py`).

RNF2: Concorrência e Multithreading: A renderização gráfica e o servidor web não devem bloquear durante o processamento matemático ou leitura de ficheiros volumosos; utilização de *background workers* em threads dedicadas.

RNF3: Rigor Matemático Sem Bibliotecas Opacas: Todo o núcleo computacional de transformações WGS-84, quaternions, Kalman/RTS, Kabsch-Umeyama e CPA deve ser implementado de raiz em NumPy/C++, usando bibliotecas externas (`pymap3d`, `scipy`) apenas para validação e testes unitários.

RNF4: Desempenho e Latência: Resposta imediata na interação com o slider temporal ($<$ 50 ms por quadro renderizado), permitindo experiência fluida a 20+ FPS no navegador.

RNF5: Testabilidade e Cobertura: Cobertura de testes unitários automatizados em `pytest`, validando invariantes geométricas ($\mathbf{R}^\top\mathbf{R} = \mathbf{I}$, $\det \mathbf{R} = 1$, ida-e-volta com erro $< 10^{-6}$ m).

4 Arquitetura de Software Proposta

A nova organização estrutural do projeto consolidado é desenhada para garantir modularidade absoluta, alta manutenibilidade e extensibilidade:

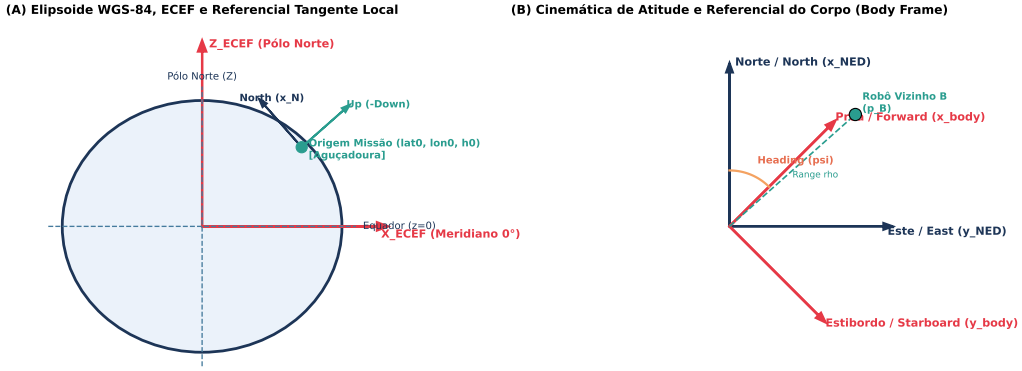


Figura 1: Fluxograma concetual da cadeia de referenciais matemáticos e processamento.

4.1 Estrutura de Ficheiros e Módulos do Novo Projeto

Abaixo apresenta-se a taxonomia de ficheiros da pasta unificada do projeto:

```
multi_robot_system/
|-- config/
|   +-- mission_config.yaml          # Parametrizacao integral da missao
|-- data/
|   |-- robot1_s1.csv                # Telemetria real INESC TEC Robot 1
|   |-- robot4_s1.csv                # Telemetria real INESC TEC Robot 4
|   +-- robot7_s1.csv                # Telemetria real INESC TEC Robot 7
|-- core/
|   |-- __init__.py
|   |-- math/
|   |   |-- geodesy.py               # WGS-84, ECEF, NED, ENU, AER (NumPy puro)
|   |   |-- rotations.py             # Quaternions, matrizes SO(3), Euler, SLERP
|   |   |-- kalman_rts.py            # Filtro de Kalman linear e Suavizador RTS
|   |   |-- umeyama.py               # Algoritmo de Kabsch-Umeyama (R, t, s, RMSE)
|   |   |-- proximity.py             # Vetores relativos, AER e calculo analitico de
    CPA
|   |   +-- interpolators.py         # Comparador Linear vs Spline Cubica vs Kalman
|   +-- data/
|       |-- odom_loader.py           # Leitor robusto de CSV ROS Odometry
|       |-- trajectory.py            # Dataclass RobotTrajectory e MissionDataset
|       +-- synchronizer.py          # Sincronizador temporal e regularizacao de
    tempo
|-- visualization/
|   |-- __init__.py
|   |-- plotly_3d_view.py            # Visualizador global 3D interativo em Plotly
|   |-- plotly_pov_view.py           # Motor de camara em 1a pessoa (POV submarino)
|   |-- hud_renderer.py              # Instrumentacao e telemetria sobreposta (HUD)
|   +-- dashboard_app.py             # Servidor web integrado (Plotly Dash/Flask)
|-- io_export/
|   |-- geojson_exporter.py          # Exportador para SIG (QGIS)
|   |-- kml_exporter.py              # Exportador para Google Earth 3D
|   +-- report_generator.py          # Exportador de metricas JSON e relatorios
|-- tests/
|   |-- test_geodesy.py              # Validacao vs pymap3d e pyproj
|   |-- test_rotations.py            # Testes de grupo SO(3) e quaternioes
|   |-- test_kalman_rts.py           # Verificacao de reducao de covariancia
|   |-- test_umeyama.py              # Teste com rotacoes e ruidos sinteticos
|   +-- test_cpa.py                  # Validacao do tempo e distancia minima CPA
|-- docs/
|   |-- especificacao_requisitos.pdf
|   +-- fundamentacao_matematica.pdf
```

```
|-- main.py                          # Ponto de entrada CLI (arranque de modos)
|-- requirements.txt                 # Dependencias Python (numpy, scipy, plotly, etc.)
+-- README.md                        # Manual de instalacao e execucao
```

Listing 1: Estrutura de pastas do projeto integrado

4.2 Fluxo de Dados e Concorrência Multithreaded

1. **Carregamento e Pré-Processamento (Thread de Inicialização)**:
   - O ficheiro `odom_loader.py` lê os CSVs em streaming de blocos vetoriais.
   - As coordenadas (φ, λ, h) são convertidas para ECEF e projetadas em NED relativamente à origem de superfície configurada.
   - O módulo `kalman_rts.py` processa as trajetórias, gerando uma estimativa suavizada e contínua dos estados $(\mathbf{p}, \mathbf{v})$ e atitudes $(\mathbf{q})$.
2. **Servidor Web e Engine de Renderização (Thread de Interface)**:
   - Um servidor leve assíncrono em Python gere as sessões do navegador.
   - O cliente web interage com um único dashboard onde um botão seletor permite comutar instantaneamente entre:
     - **Vista Global 3D**: Visão panorâmica estilo `trajectories.html` com todos os submarinos, malha de fundo e batimetria.
     - **Vista POV 1.ª Pessoa**: Câmara acoplada ao bico de cada drone com HUD dinâmico e mira para os robôs circundantes.
3. **Motor de Análise Cinemática em Background**:
   - Os cálculos de proximidade, CPA e matriz de distâncias relativas correm em estruturas pré-vetorizadas em memória, permitindo inspeção instantânea ao movimentar a barra temporal.

5 Roteiro de Implementação e Validação

A consolidação do projeto seguirá cinco marcos metodológicos:

Fase	Atividades Principais	Entregável Chav
Fase 1	Esclarecimento dos requisitos com as 50 perguntas e validação teórica.	Questionário respo
Fase 2	Criação da estrutura de pastas modular e migração do núcleo matemático.	Pacote `core/math`
Fase 3	Implementação de Kalman/RTS, Kabsch-Umeyama e análise CPA.	Scripts com valida
Fase 4	Unificação das vistas Plotly 3D e POV em primeira pessoa com threading.	Dashboard web int
Fase 5	Exportadores KML/GeoJSON, documentação final em LaTeX e revisão geral.	Projeto integrado

Tabela 2: Cronograma metodológico de integração.